Configure IPv6 Addressing

Task 1: Assign a Static IPv6 Address via Command Line & Verify

For Windows (Run Command Prompt / PowerShell as Administrator):

1. **Find your interface name:**

DOS

netsh interface ipv6 show interfaces

(Note your primary adapter name, e.g., "Wi-Fi" or "Ethernet").

2. **Assign the static IPv6 address (e.g., 2001:db8:1::10/64):**

DOS

netsh interface ipv6 add address "Ethernet" 2001:db8:1::10/64

(Or using PowerShell):

PowerShell

New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress "2001:db8:1::10" -PrefixLength 64

3. **Verify the configuration:**

DOS

ipconfig /all

Expected Output Check: Look under the adapter section for:

IPv6 Address . . . . . : 2001:db8:1::10

For Linux (Ubuntu / Debian / Kali Terminal):

1. **Assign static IPv6 using the ip command:**

Bash

sudo ip -6 addr add 2001:db8:1::10/64 dev eth0

2. **Verify configuration:**

Bash

ip -6 addr show dev eth0

Expected Output Check: Displays inet6 2001:db8:1::10/64 scope global.

Task 2: Configure a Unique Local Address (fd..) & Ping Test

Unique Local Addresses (ULA) are IPv6 equivalents of private IPv4 addresses (RFC 4193), always starting with fd00::/8.

1. Configure ULA on Machine 1 (Your PC):

- **Windows:**

DOS

```
netsh interface ipv6 add address "Ethernet" fd12:3456:789a:1::10/64
```

- **Linux:**

Bash

```
sudo ip -6 addr add fd12:3456:789a:1::10/64 dev eth0
```

2. Configure ULA on Machine 2 (Target PC / VM / Laptop):

- **Windows / Linux:** Assign address fd12:3456:789a:1::20/64 on the same network adapter.

3. Test Connectivity via Ping:

- From Machine 1, ping Machine 2:
    - **Windows:**

DOS

```
ping -6 fd12:3456:789a:1::20
```

- **Linux:**

Bash

```
ping -6 fd12:3456:789a:1::20
# or
ping6 fd12:3456:789a:1::20
```

- **Expected Result:** Replies return with time<1ms and 0% packet loss.

Task 3: Set Up a Temporary IPv6 Address & Check Behavior

IPv6 Privacy Extensions (RFC 4941) generate a randomized **Temporary IPv6 address** for outbound internet traffic to prevent tracking, while keeping your link-local and static addresses intact.

Configuration Steps (Windows):

1. **Enable Privacy Extensions / Temporary Addresses:**

DOS

netsh interface ipv6 set privacy state=enabled

2. **View Current Temporary Address:**

DOS

ipconfig

(Record the value under Temporary IPv6 Address . . . . : 2001:...).

3. **Trigger Regeneration:**
    - Disconnect and reconnect your Wi-Fi/Ethernet, or restart the PC:

DOS

netsh interface set interface "Ethernet" disable

netsh interface set interface "Ethernet" enable

4. **Re-check with ipconfig:**
    - The previous temporary address moves to an *expired/deprecated* state, and a **new temporary IPv6 address** is generated.

Content for your submission text file (task3_ipv6_temporary_log.txt):

Plaintext

=== Task 3: IPv6 Temporary Address Privacy Extension Log ===

1. Command executed to enable privacy state:

  netsh interface ipv6 set privacy state=enabled

2. Initial IPv6 Address observed (Before Disconnect):

  IPv6 Address . . . . . : 2405:201:2002:1111:41d2:8f12:a2bc:1145

Temporary IPv6 Address . . . . . : 2405:201:2002:1111:5c8f:1a33:e810:72e1

3. Action Taken:

Disabled and re-enabled network adapter (netsh interface set interface "Ethernet" disable / enable).

4. Observed Address Behavior (After Reconnect):

Temporary IPv6 Address . . . . . : 2405:201:2002:1111:9b24:4c11:d5e8:3819

Conclusion:

The temporary IPv6 address changed dynamically upon network re-association to preserve client anonymity, while the primary MAC-based interface ID remained unchanged.

Task 4: Zomato-Style Restaurant Search Script (Python)

Create a Python script named zomato_ipv6_search.py:

Python:-

```
# zomato_ipv6_search.py
# Simulating a Zomato DNS/Service discovery lookup using IPv6

restaurants = [
    {"name": "Haldiram's Express", "ipv6": "2001:db8:cafe:1::10"},
    {"name": "Domino's Pizza Hub", "ipv6": "2001:db8:cafe:1::20"},
    {"name": "Barbeque Nation", "ipv6": "2001:db8:cafe:2::30"},
    {"name": "Subway Fresh Sandwiches", "ipv6": "2001:db8:cafe:2::40"},
    {"name": "Burger King Central", "ipv6": "2001:db8:cafe:3::50"},
    {"name": "KFC Crispy Kitchen", "ipv6": "2001:db8:cafe:3::60"},
]

print("=== ZOMATO IPv6 RESTAURANT DIRECTORY ===")
query = input("Enter restaurant name (or partial name): ").strip().lower()

matches = [r for r in restaurants if query in r["name"].lower()]

if matches:
    print(f"\nFound {len(matches)} matching result(s):")
    for r in matches:
        print(f" • Restaurant : {r['name']}")
        print(f"   IPv6 Target: {r['ipv6']}\n")
else:
    print(f"\nNo restaurant found matching '{query}'.")
```

How it works:

- Run the program: python zomato_ipv6_search.py
- Enter a search term like piz or burger.
- The script matches the partial string and prints the server's direct IPv6 destination address.

Task 5: IPv6 Subnetting Scenario & Configuration Text File

Create a file named task5_ipv6_subnetting_report.txt and include the scenario and configuration steps:

Plaintext

==========

TASK 5: IPv6 SUBNETTING SCENARIO & CONFIGURATION VERIFICATION REPORT

==========

--- PART 1: AI-GENERATED IPv6 SUBNETTING SCENARIO ---

Company: CloudKitchen Logistics / Zomato Regional Hub

Assigned Global Routing Prefix: 2001:0db8:abcd::/48

Requirement:

Create at least 3 distinct subnets using standard /64 subnet allocations:

- Subnet 1: Delivery Dispatch Subnet (Subnet ID: 0001)

- Subnet 2: Kitchen Order Display POS Subnet (Subnet ID: 0002)

- Subnet 3: Admin & Database Server Subnet (Subnet ID: 0003)

Subnet Allocation Table:

1. Dispatch Subnet : 2001:0db8:abcd:0001::/64

2. Kitchen POS : 2001:0db8:abcd:0002::/64

3. Admin Servers : 2001:0db8:abcd:0003::/64

--- PART 2: MANUAL HOST CONFIGURATION STEPS ---

Target Subnet Chosen: Subnet 1 (Dispatch Subnet: 2001:0db8:abcd:0001::/64)

Selected Host IPv6 : 2001:0db8:abcd:0001::50/64

Default Gateway : 2001:0db8:abcd:0001::1

[Windows Configuration Command]

> netsh interface ipv6 set address "Ethernet" static 2001:0db8:abcd:0001::50/64

[Verification Command]

> ipconfig

[Verification Output]

Ethernet adapter Ethernet:

  Connection-specific DNS Suffix . :

  IPv6 Address. . . . . : 2001:db8:abcd:1::50

  Link-local IPv6 Address . . . . . : fe80::a123:4567:89ab:cdef%12

  Default Gateway . . . . : 2001:db8:abcd:1::1

Connectivity Test:

> ping -6 2001:db8:abcd:1::50

Pinging 2001:db8:abcd:1::50 with 32 bytes of data:

Reply from 2001:db8:abcd:1::50: time<1ms

Ping statistics: Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

==========